

EIE4113 Wireless and Mobile Systems

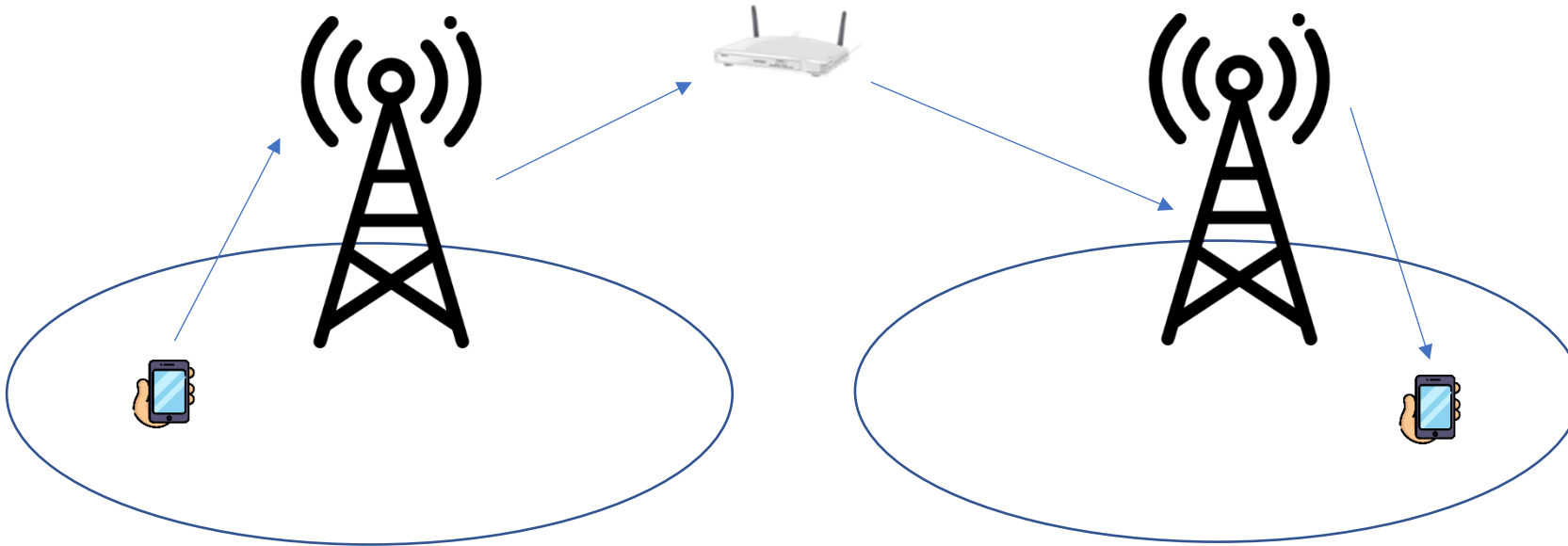
Routing Algorithm

- Week 6

Routing is Necessary for Wireless Network

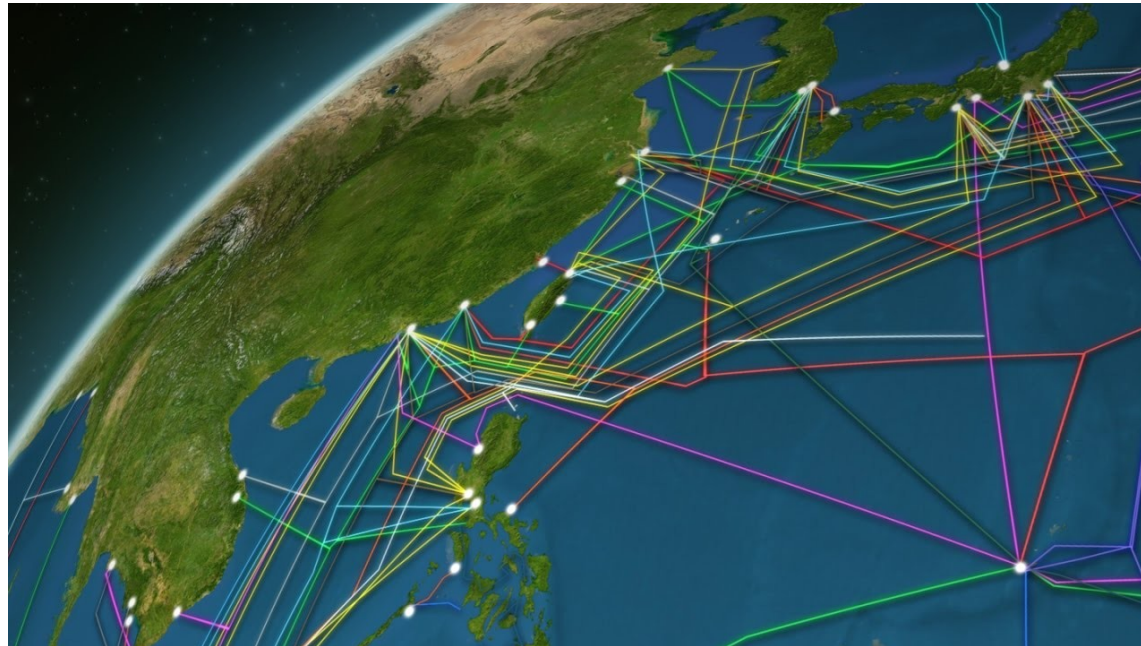
❑ When the transmitter and receiver are not in the same cell, base stations should talk with each other

- Base stations are connected by fiber cables (backhaul network)



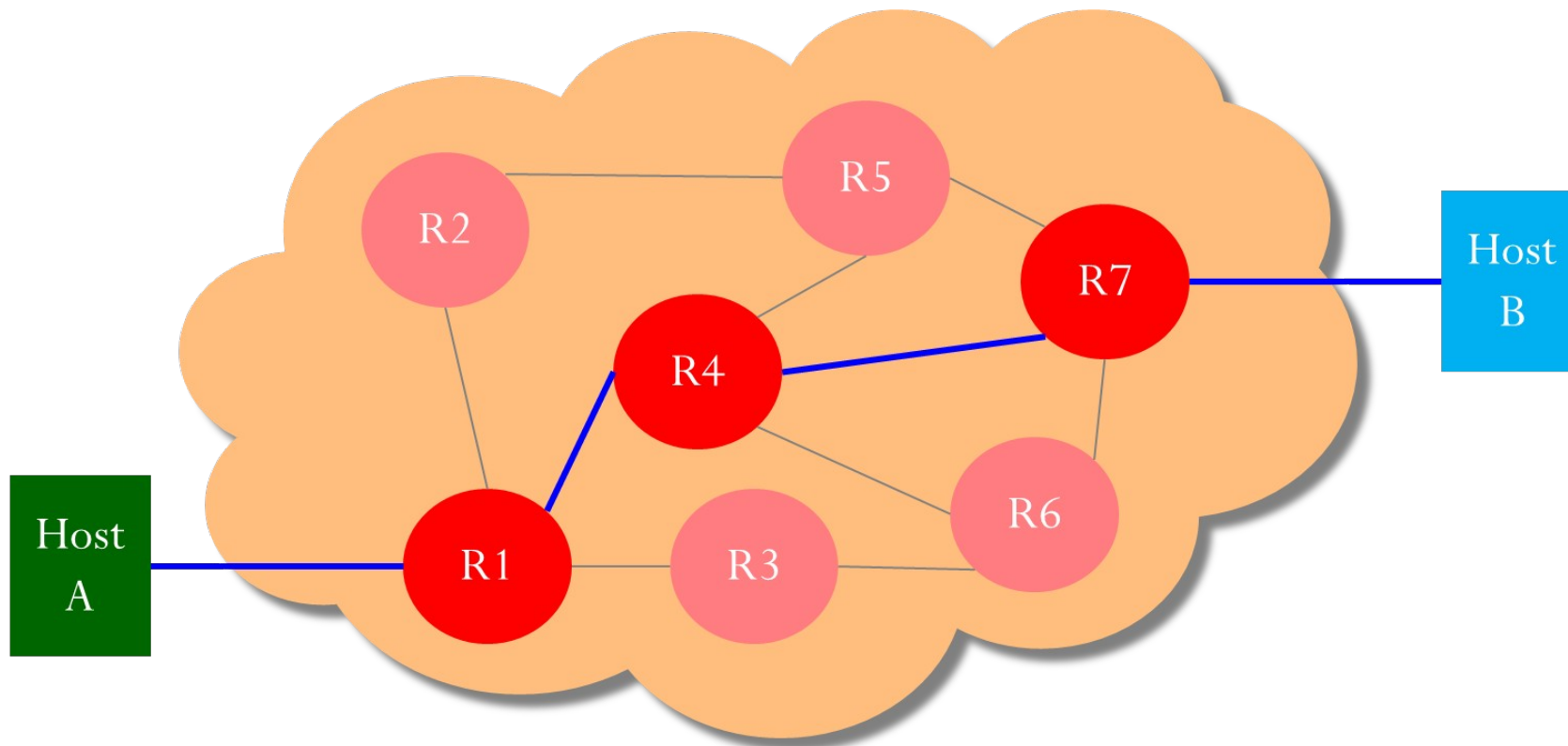
Routing is Necessary for Wireless Network

- ❑ When you are at Hong Kong, and your friend is at London
 - Your signal is sent to a base station
 - Signal is then routed to London, and then a base station there serves your friend



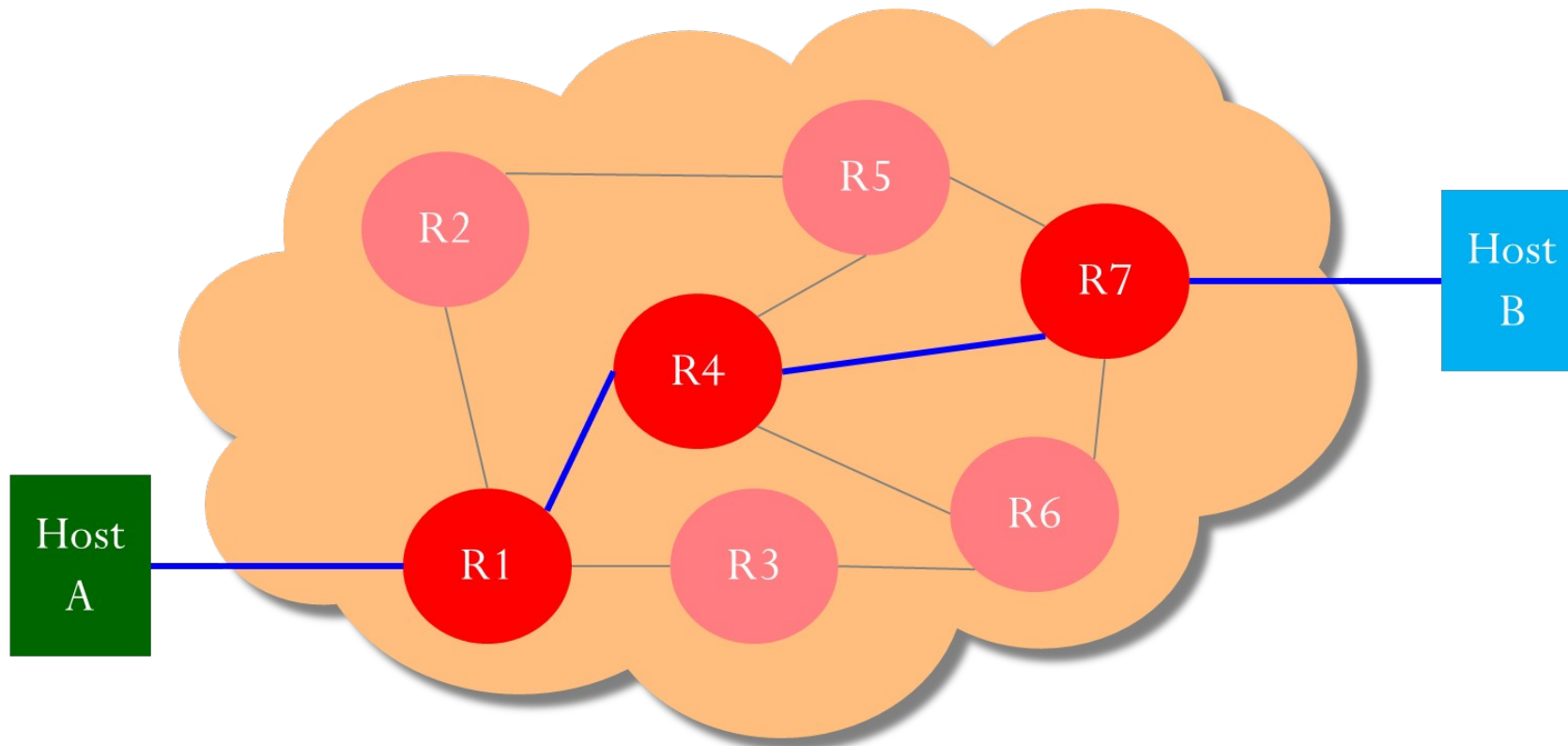
Routing in Fiber Network

❑ When transmitter and receiver is connected via a lot of routers, it is hard to find a good routing strategy



Routing in Fiber Network

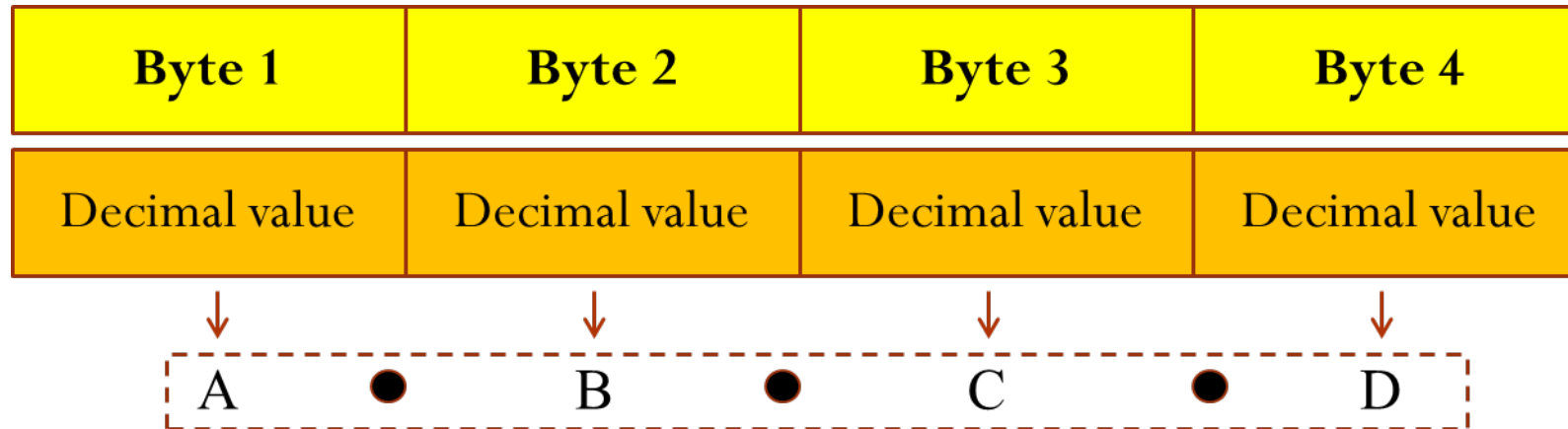
❑ When transmitter and receiver is connected via a lot of routers, it is hard to find a good routing strategy



IP Address

❑ IP addresses are 32-bit long, usually expressed in the dotted-decimal notation

A.B.C.D



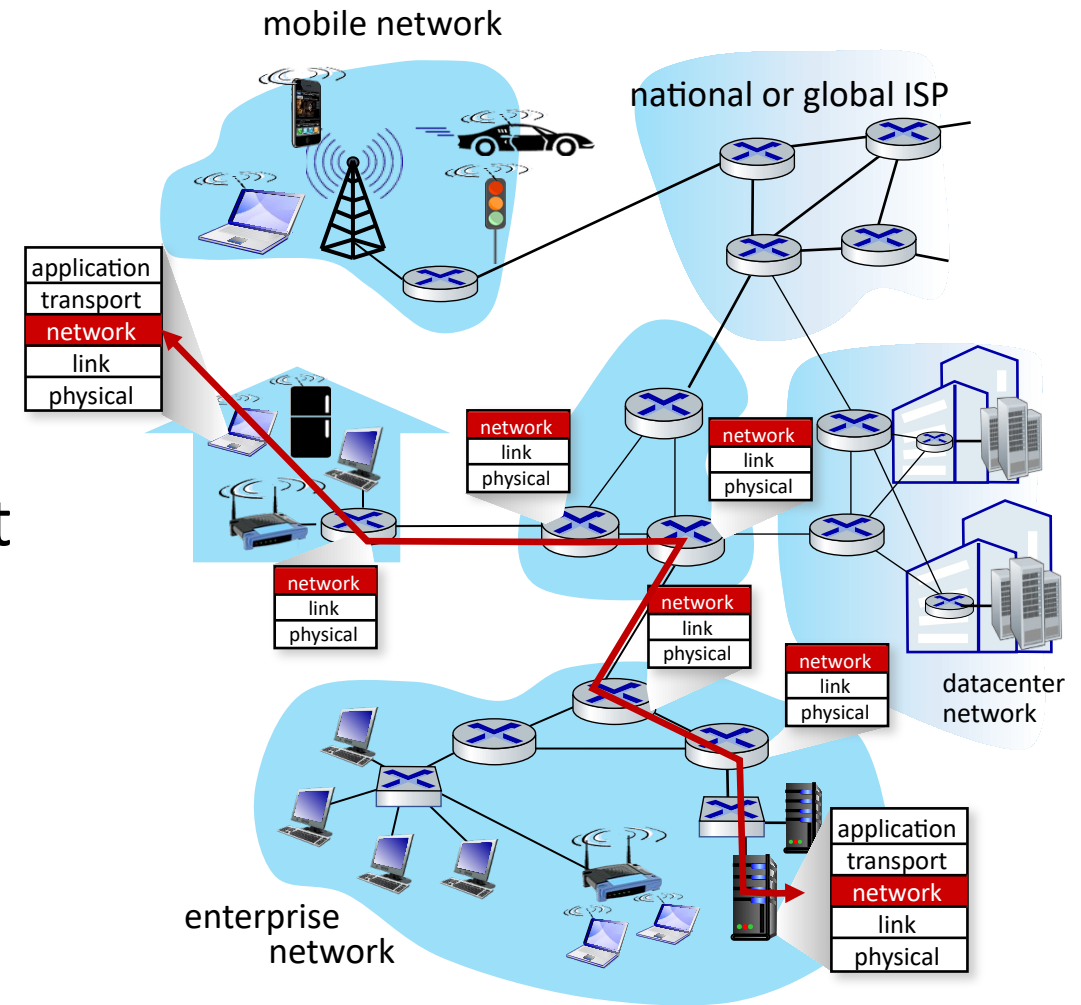
❑ Example: 152.123.1.2

10011000 10000100 00000001 00000010

Routing protocols

Routing protocol goal: determine “good” paths (equivalently, routes), from sending hosts to receiving host, through network of routers

- **path:** sequence of routers packets traverse from given initial source host to final destination host
- **“good”:** least “cost”, “fastest”, “least congested”
- routing: a “top-10” networking challenge!



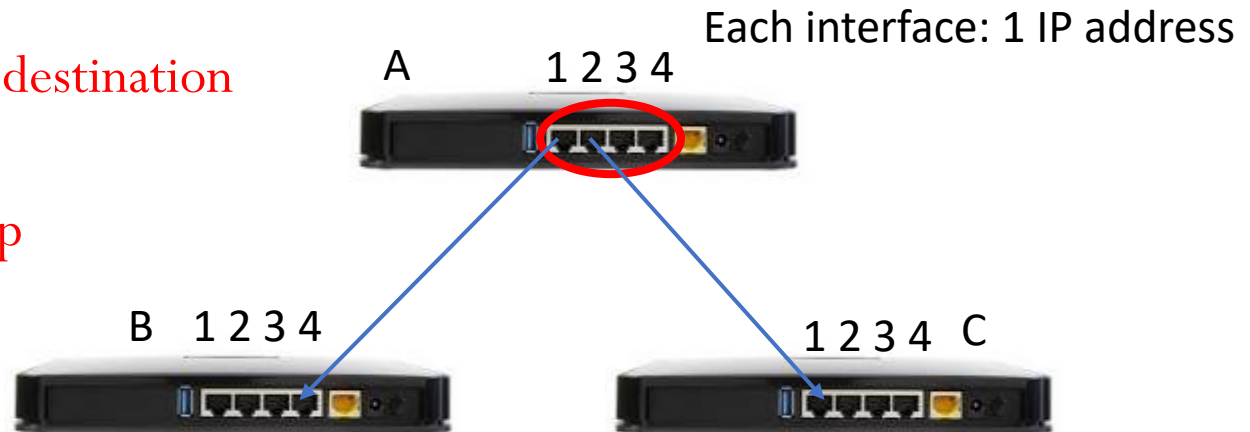
Routing tables

- Within each router, there is a **routing table** specifying how to route packets to the destination.
- The rules that used to build a routing table is called a **routing algorithm**.

■ Example:

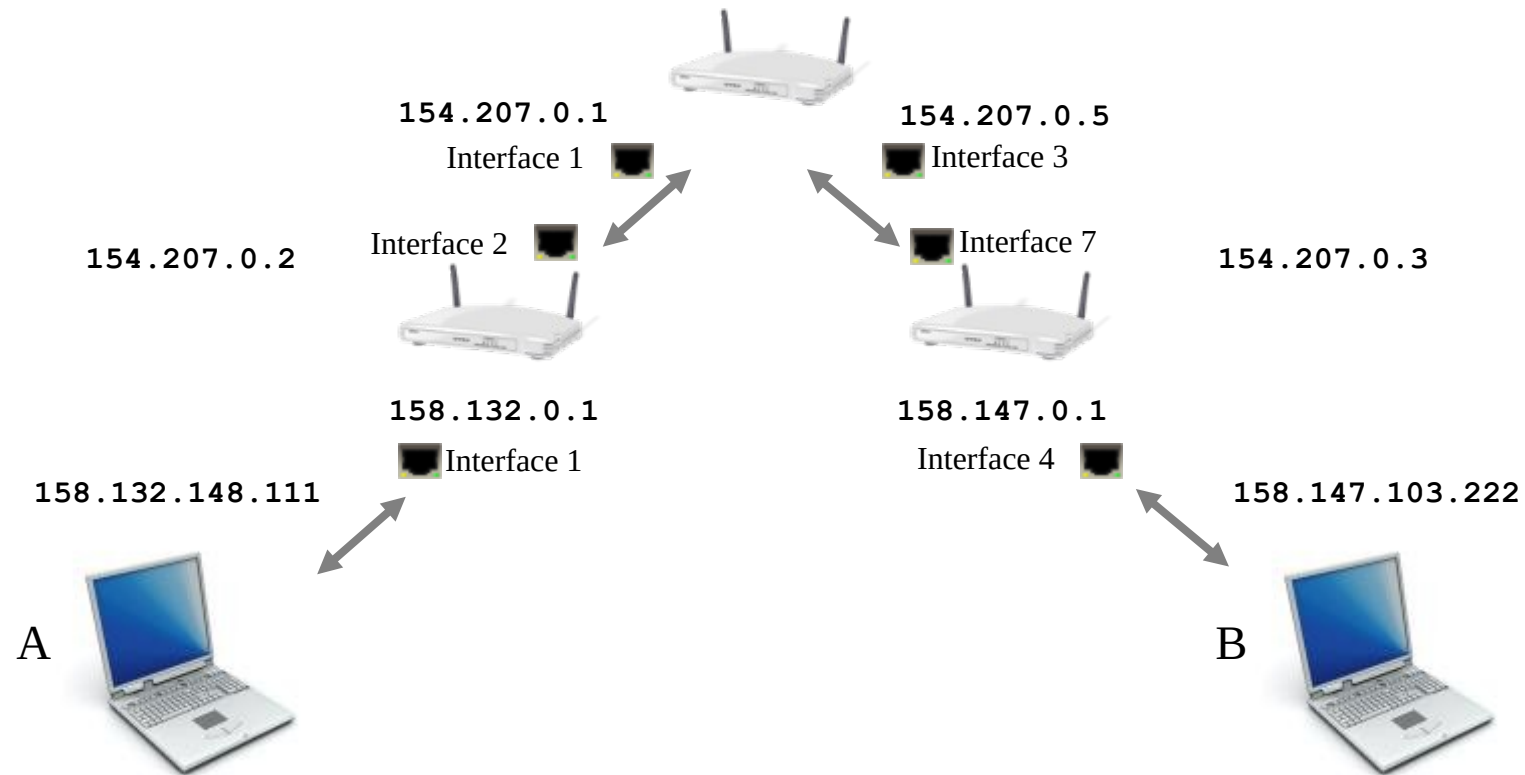
Destination (IP)	Gateway (IP)	Interface (Index)
IP of Network B	IP of B's Interface 4	1
IP of Network C	IP of C's Interface 1	2

- Destination: IP address of **final destination**
- Outgoing interface: index
- Gateway: IP address of **next hop**



Routing example

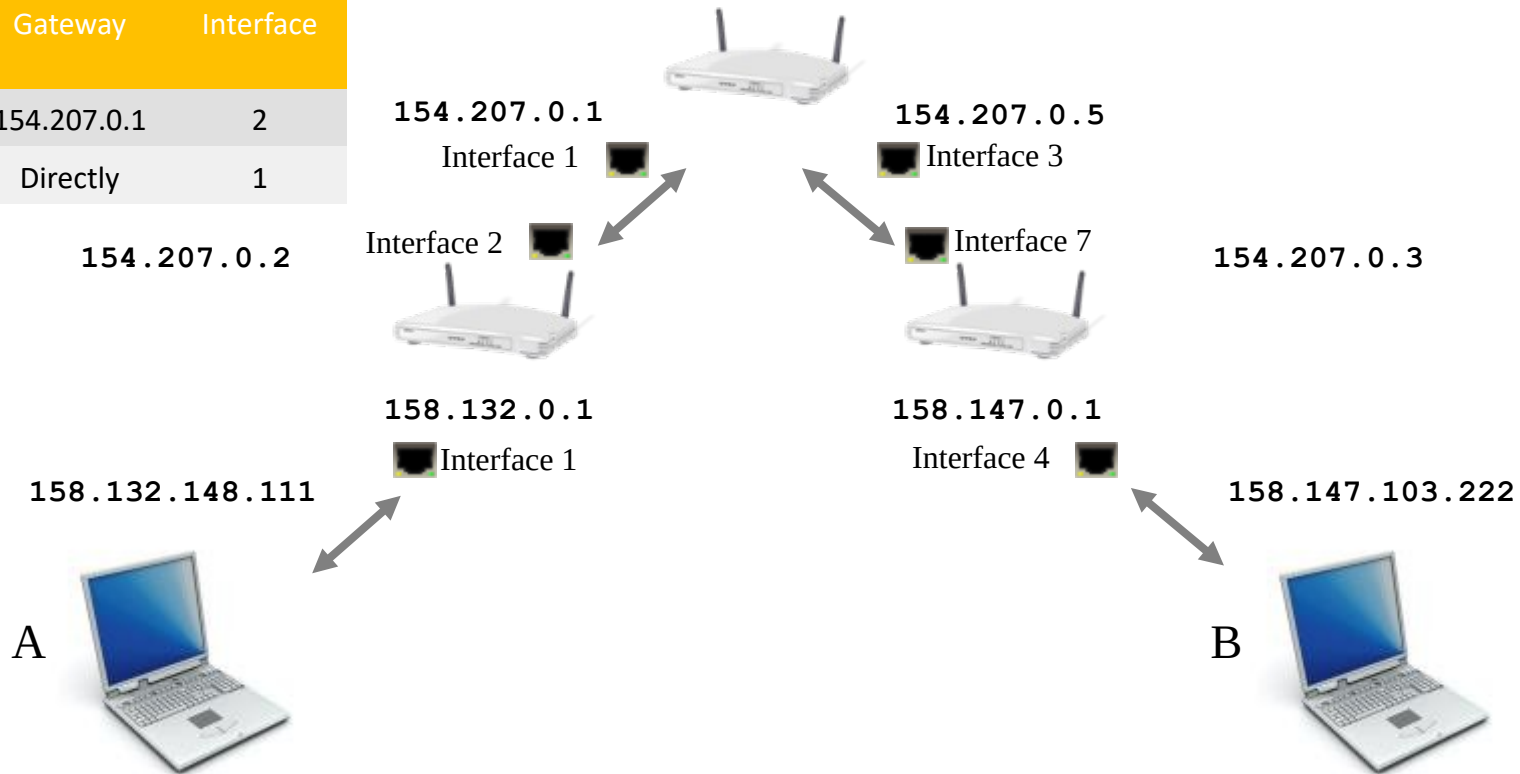
- Two networks are connected through three routers



Routing example

■ Routing table at each router

Network Destination	Gateway	Interface
158.147.0.0	154.207.0.1	2
158.132.0.0	Directly	1



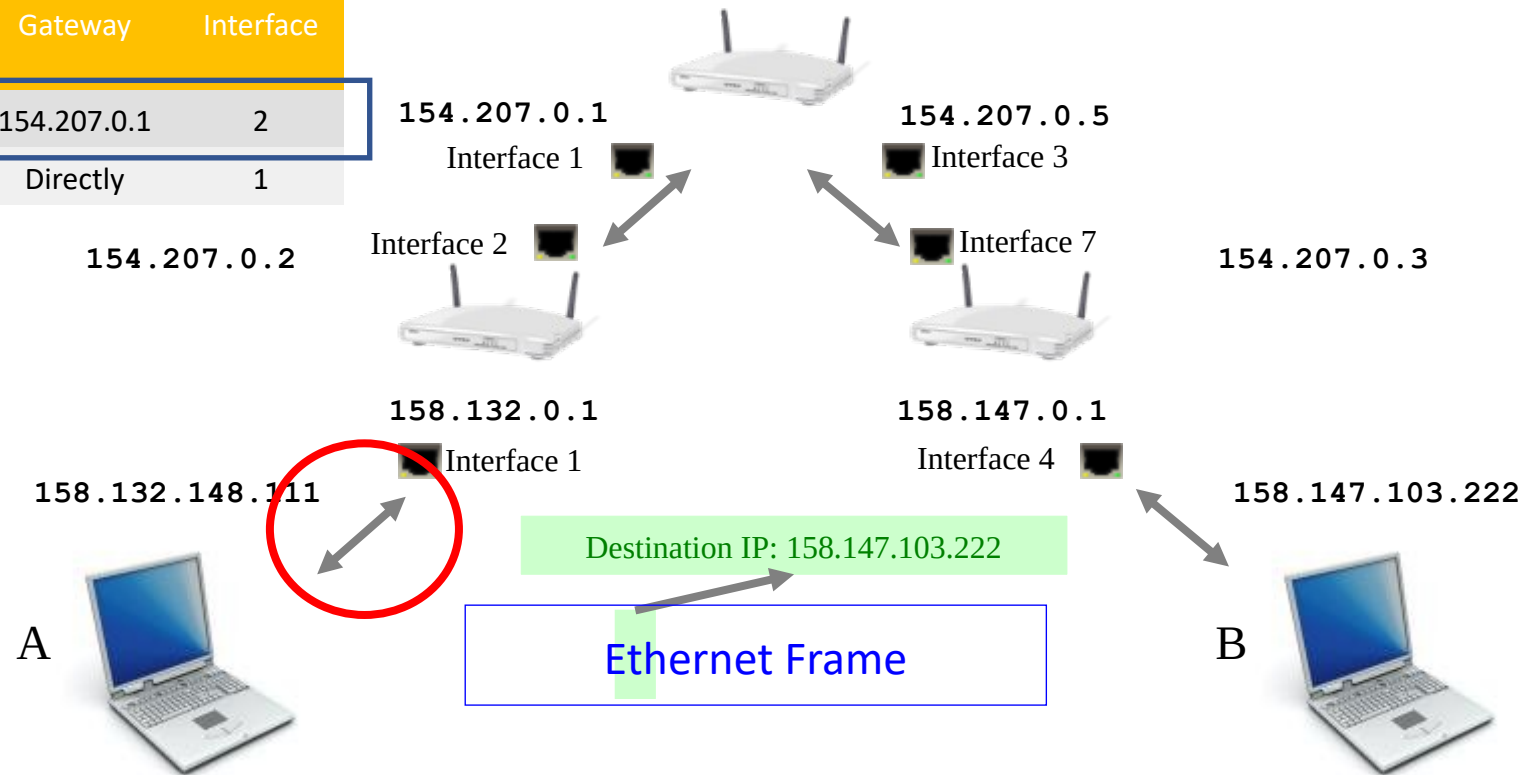
Network Destination	Gateway	Interface
158.147.0.0	154.207.0.3	3
158.132.0.0	154.207.0.2	1

Network Destination	Gateway	Interface
158.147.0.0	Directly	4
158.132.0.0	154.207.0.5	7

Routing example

■ Host A to Host B - IP and Ethernet Addresses

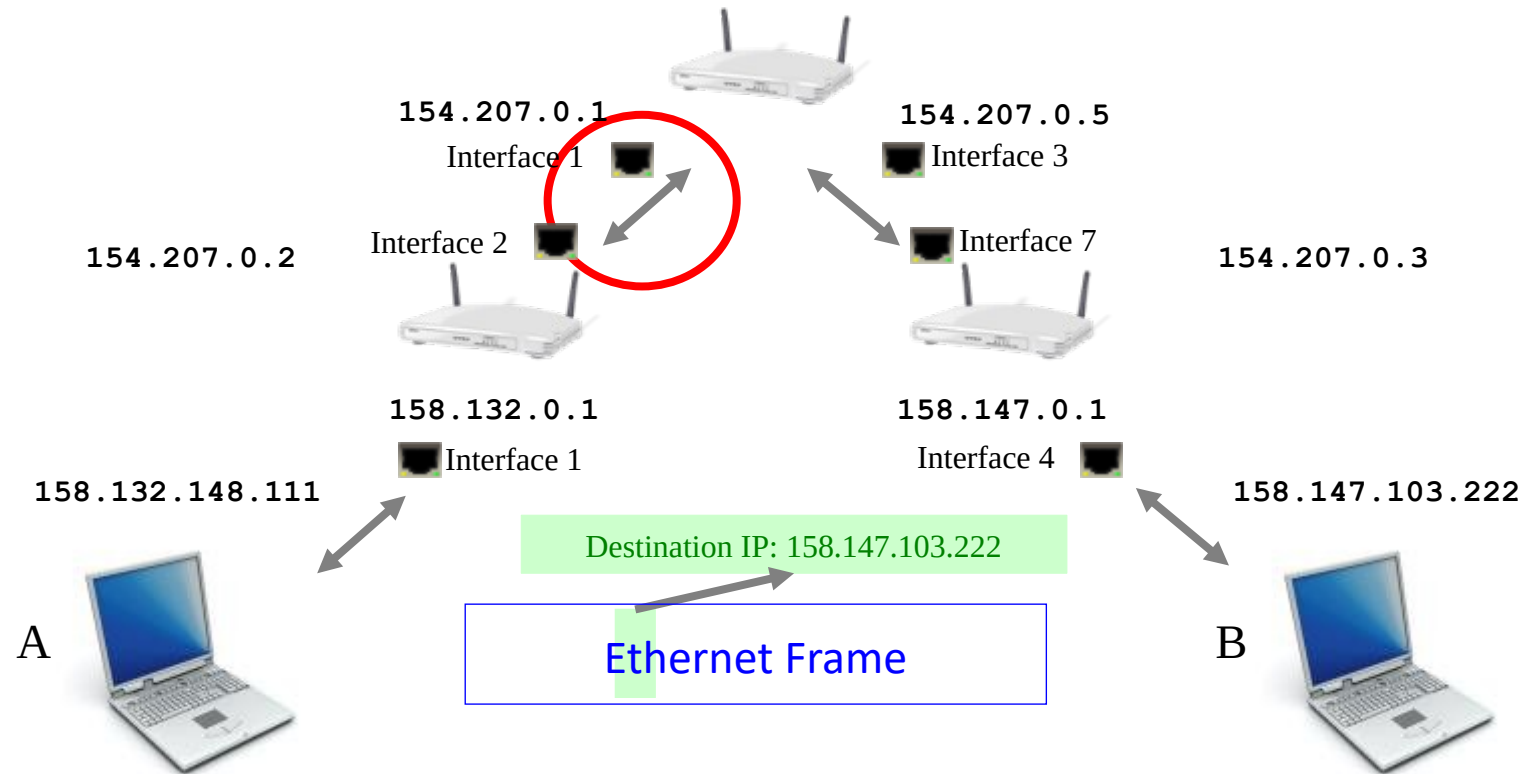
Network Destination	Gateway	Interface
158.147.0.0	154.207.0.1	2
158.132.0.0	Directly	1



Routing example

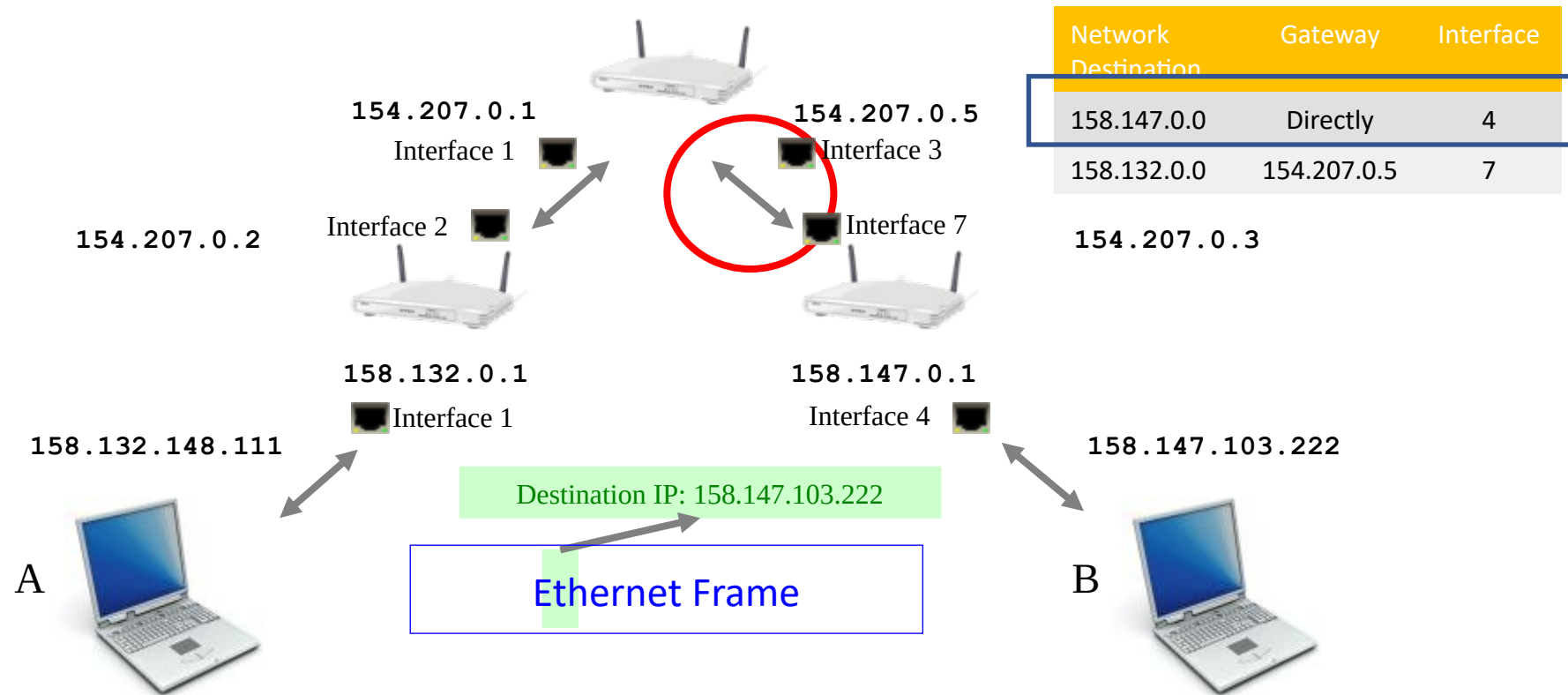
Network Destination	Gateway	Interface
158.147.0.0	154.207.0.3	3
158.132.0.0	154.207.0.2	1

- Host A to Host B (cont'd) - IP and Ethernet Addresses



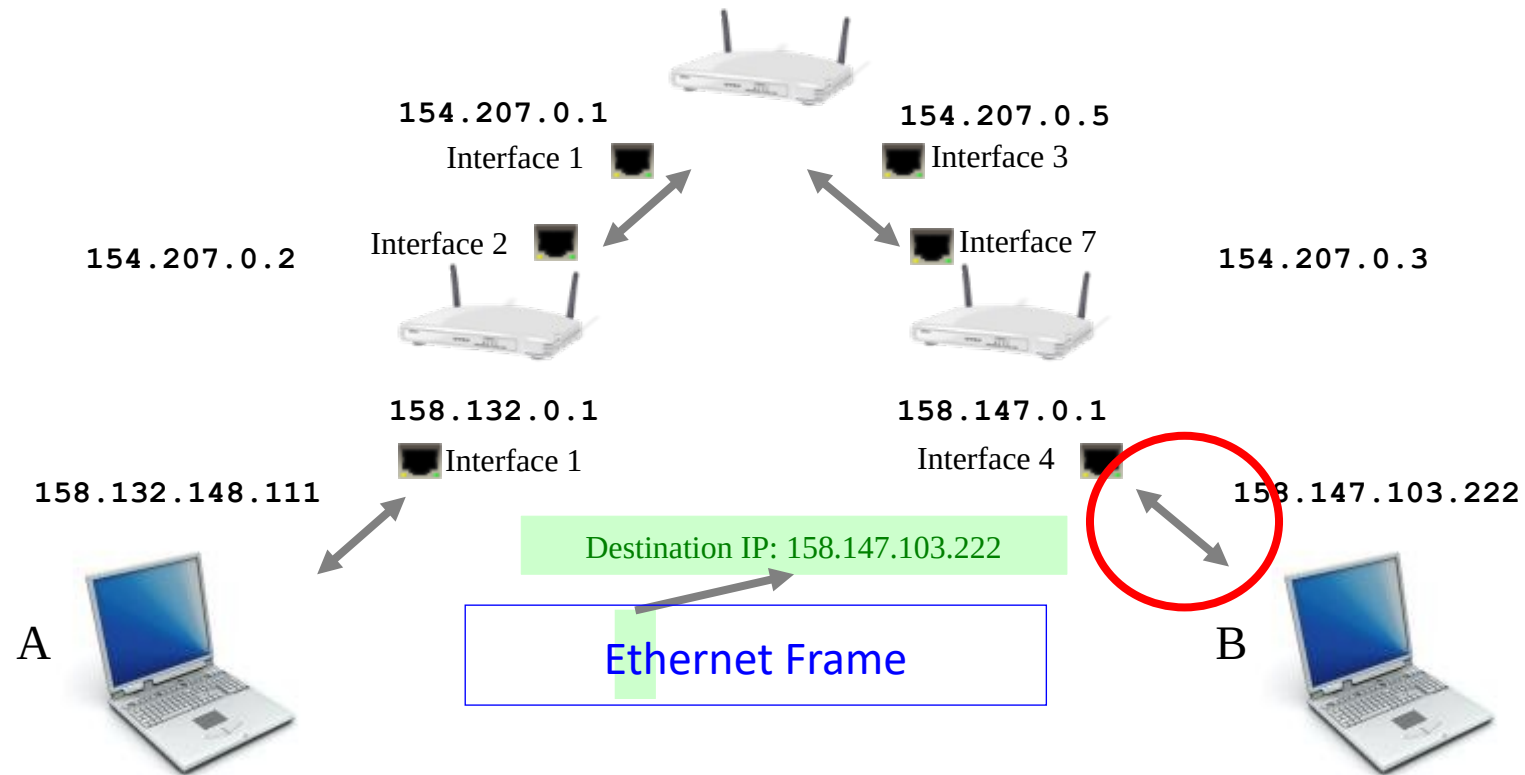
Routing example

- Host A to Host B (cont'd) - IP and Ethernet Addresses

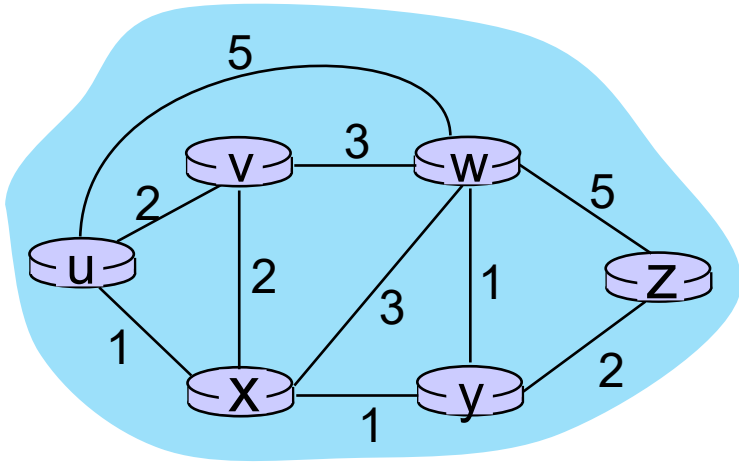


Routing example

- Host A to Host B (cont'd) - IP and Ethernet Addresses



Graph abstraction: link costs



$c_{a,b}$: cost of *direct* link connecting a and b

e.g., $c_{w,z} = 5$, $c_{u,z} = \infty$

cost defined by network operator:
could always be 1, or inversely related
to bandwidth, or related to congestion

graph: $G = (N, E)$

N (node): set of routers = $\{ u, v, w, x, y, z \}$

E (edge): set of links = $\{ (u,v), (u,x), (u,w), (v,x), (v,w), (x,w), (x,y), (w,y), (w,z), (y,z) \}$

Dijkstra's link-state routing algorithm

- **centralized:** network topology, link costs known to *all* nodes
 - accomplished via “link state broadcast”
 - all nodes have same info
- computes least cost paths from one node (“source”) to all other nodes
 - gives *forwarding table* for that node
- **iterative:** after k iterations, know least cost path to k destinations

notation

- $c_{a,b}$: direct link cost from node a to b ; $= \infty$ if not direct neighbors
- $D(b)$: *current* estimate of cost of least-cost-path from source to destination b
- $p(b)$: predecessor node along path from source to b
- N' : set of nodes whose least-cost-path *definitively* known

Dijkstra's link-state routing algorithm

1 *Initialization:*

2 $N' = \{u\}$

/ compute least cost path from u to all other nodes */*

3 for all nodes b

4 if b adjacent to u

/ u initially knows direct-path-cost only to direct neighbors */*

5 then $D(b) = c_{u,b}$

/ but may not be minimum cost!*

**/*

6 else $D(b) = \infty$

7

8 *Loop*

9 find a not in N' such that $D(a)$ is a minimum

10 add a to N'

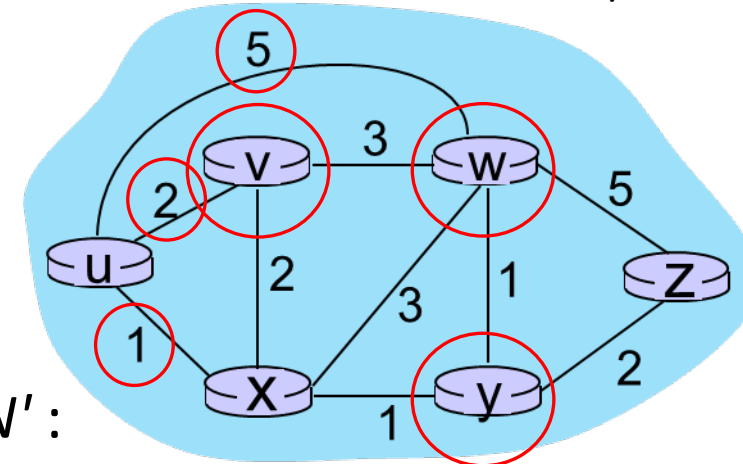
11 update $D(b)$ for all b adjacent to a and not in N' :

12 $D(b) = \min (D(b), D(a) + c_{a,b})$

13 */* new least-path-cost to b is either old least-cost-path to b or known*

14 *least-cost-path to a plus direct-cost from a to b */*

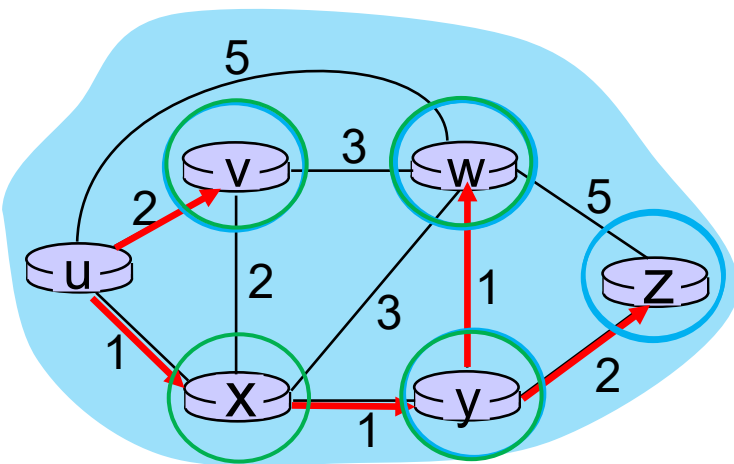
15 *until all nodes in N'*



Dijkstra's algorithm: an example

why?

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$	
0	u	2, u	5, u	1, u	∞	∞	one-hop cost
1	ux	2, u	4, x		2, x	∞	one-hop and two-hop cost
2	uxy	2, u	3, y			4, y	
3	uxyv		3, y			4, y	
4	uxyvw					4, y	
5	uxyvwz						



Initialization (step 0): For all a : if a adjacent to then $D(a) = c_{u,a}$

find a not in N' such that $D(a)$ is a minimum

add a to N'

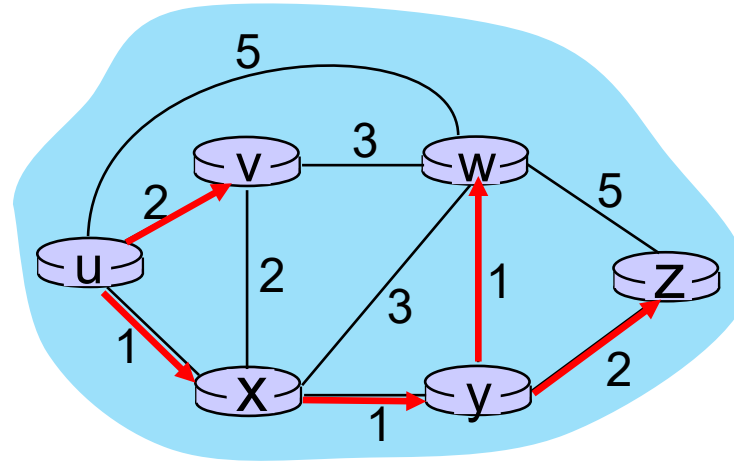
update $D(b)$ for all b adjacent to a and not in N' :

$$D(b) = \min (D(b), D(a) + c_{a,b})$$

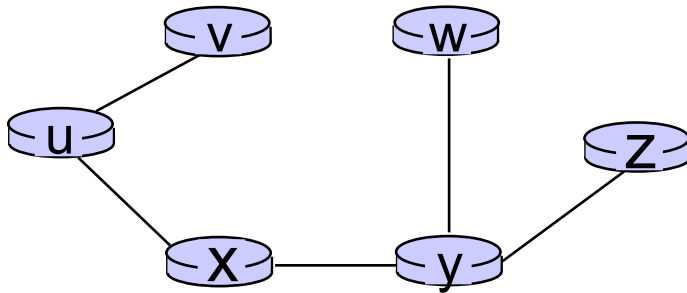
Other than u and x , the one-hop and two-hop costs to other routers are always equal to or larger than 2 (the minimum one-hop and two-hop cost). Hence, it is not possible to find better path to y via the other routers.

Is it possible to visit v, w, y , or z with lower cost in two hops via x ?

Dijkstra's algorithm: an example



resulting least-cost-path tree from u:



resulting forwarding table in u:

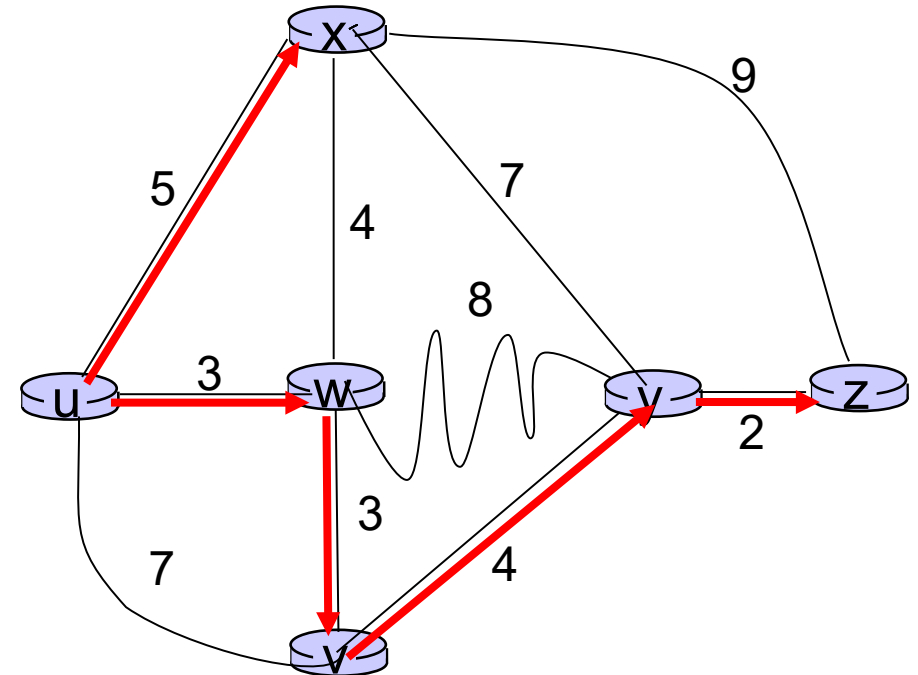
destination	outgoing link
v	(u,v)
x	(u,x)
y	(u,x)
w	(u,x)
z	(u,x)

route from u to v directly

route from u to all other destinations via x

Dijkstra's algorithm: another example

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	7, u	3, u	5, u	∞	∞
1	uw	6, w		5, u	11, w	∞
2	uwx	6, w			11, w	14, x
3	uwxv				10, v	14, x
4	uwxvy					12, y
5	uwxvyz					



notes:

- construct least-cost-path tree by tracing predecessor nodes
- ties can exist (can be broken arbitrarily)

Dijkstra's algorithm: discussion

algorithm complexity: n nodes

- each of n iteration: need to check all nodes, b , not in N'
- $n(n+1)/2$ comparisons: $O(n^2)$ complexity
- more efficient implementations possible: $O(n \log n)$

message complexity:

- each router must *broadcast* its link state information to other n routers
- efficient (and interesting!) broadcast algorithms: $O(n)$ link crossings to disseminate a broadcast message from one source
- each router's message crosses $O(n)$ links: overall message complexity: $O(n^2)$

Mid-Term Test

- **Mid-Term Test: 10%**

- Time: Wednesday of Week 7, 9:50 – 11:20
- Location: HJ202
- Closed-book exam
- Week 1 – Week 6